



**HỌC VIỆN
NGÂN HÀNG**
BANKING ACADEMY OF VIETNAM

CHƯƠNG 5 NGÔN NGỮ T-SQL

KHOA CÔNG NGHỆ THÔNG TIN VÀ KINH TẾ SỐ



- Giới thiệu về T-SQL
- Một số câu lệnh điều khiển
- Hàm
- Thủ tục
- Tạo bất dữ liệu – Trigger



- T-SQL (Transact-SQL) là một tập hợp các tiện ích lập trình mở rộng của Sybase và Microsoft bổ sung một số tính năng từ SQL, bao gồm kiểm soát giao dịch, xử lý ngoại lệ và lỗi, xử lý hàng và các biến được khai báo.
- T-SQL chứa lập trình thủ tục và biến cục bộ, trong khi SQL thì không



- **Một giao dịch (transaction) là một đơn vị công việc:**
 - Nếu giao dịch thành công, tất cả các thao tác dữ liệu trong quá trình giao dịch sẽ được thực hiện và được lưu vào cơ sở dữ liệu.
 - Nếu một giao dịch gặp lỗi và phải hủy thì tất cả các sửa đổi dữ liệu sẽ bị xóa.



■ SQL Server hoạt động ở các chế độ giao dịch sau:

- **Autocommit**: Mỗi tuyên bố riêng lẻ là một giao dịch.
- **Explicit**: Mỗi giao dịch được bắt đầu tường minh bằng câu lệnh BEGIN TRANSACTION và kết thúc tường minh bằng câu lệnh COMMIT hoặc ROLLBACK.
- **Implicit**: Một giao dịch mới được ngầm bắt đầu khi giao dịch trước đó hoàn thành

1. Giới thiệu về T-SQL



- **TRANSACTION:** Tất cả các câu lệnh trong một giao dịch phải được xử lý như một đơn vị công việc.
- **ROLL BACK:** Khi một tiến trình bị bẻ gãy ở giữa một giao dịch, như kết quả của việc reboot, hệ thống bị vỡ, thì DBMS phải đưa database về trạng thái tồn tại trước khi giao dịch được bắt đầu.
- **COMMIT:** Nếu tất cả câu lệnh trong một giao dịch đã được thực hiện thành công.
- **RAISERROR** (thông báo lỗi, mức độ nghiêm trọng (0-25), trạng thái (1-127))



Chú thích:

- Chú thích dòng đơn

- Viết chú thích một dòng đơn ở đây

- Chú thích nhiều dòng

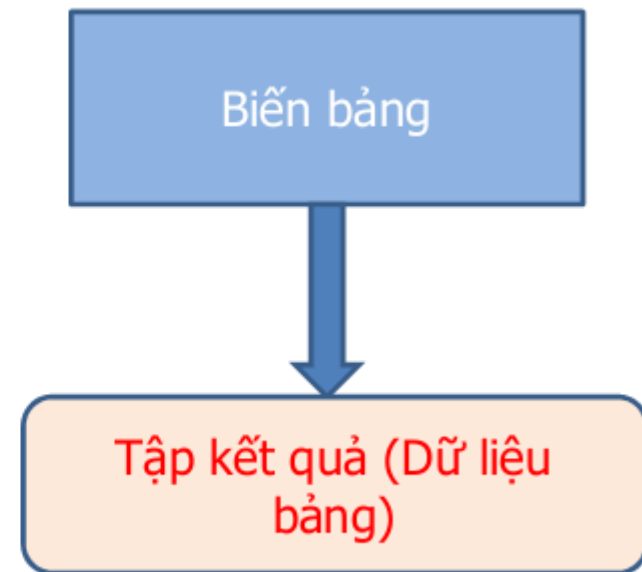
- /*

- Chú thích nhiều dòng

- */



■ Biến: Hai loại biến





Biến vô hướng:

■ Khai báo:

```
DECLARE @<tên biến 1> <kiểu dữ liệu> [,  
      @<tên biến 2> <kiểu dữ liệu>]...
```

■ Gán giá trị:

- Trực tiếp (gán giá trị cho 1 biến)

```
SET @<tên biến> = <biểu thức>
```

- Gián tiếp (gán giá trị cho ≥ 1 biến)

```
SELECT @<tên biến> = <biểu thức cột> From
```



Các phép toán:

- Số học: +, -, *, /, % (mod)
- So sánh: =, <>, >, <, <=, >=
- Logic: AND, OR, NOT, ALL, SOME, ANY, BETWEEN, EXISTS, IN, LIKE
- Ghép chuỗi (+): Dùng để ghép hai chuỗi với nhau thành một chuỗi (dùng với các kiểu dữ liệu chuỗi)



Biến bảng:

■ Khai báo:

```
DECLARE @<tên_bảng> TABLE (  
    <tên_cột_1> <kiểu_dữ_liệu>  
    [, <tên_cột_2> <kiểu_dữ_liệu>]...  
)
```

■ Biến bảng sử dụng giống như bảng SQL

- Có thể sử dụng lệnh INSERT, UPDATE, DELETE, SELECT
- Ngoại lệ: **không thể sử dụng lệnh SELECT INTO**



Hiển thị giá trị:

■ Cú pháp: **PRINT** @tên biến

■ Khi kết hợp với chuỗi, phải đổi biến sang kiểu chuỗi bằng hàm CAST hoặc CONVERT

PRINT ‘thông báo’ [+ **CAST**(@biến AS char(n))/
CONVERT(char(n),@biến]



- **Ví dụ:** Cho biết ngày sinh của sinh viên trẻ nhất

```
Declare @NS date
Select @NS = MAX(Ngsinh)
From Sinh_Vien
Print 'ngày sinh của sinh viên trẻ nhất là:' + CAST(@NS as char(10))
```

Messages

Warning: Null value is eliminated by an aggregate or other SET operation.

ngày sinh của sinh viên trẻ nhất là:1993-10-12



Lệnh điều khiển – rẽ nhánh:

■ Cú pháp:

```
IF expression
    BEGIN
        sql_statements
    END
[ELSE
    BEGIN
        sql_statements
    END]
```

- Nếu thực thi 2 hoặc nhiều câu lệnh trong mệnh đề IF hoặc ELSE cần bao các lệnh trong khối **BEGIN...END**

2. Một số câu lệnh điều khiển



Ví dụ:

```
Declare @d float
```

```
Select @d=Diem From Ket_qua Where (MaSV=2)and  
(MaMH='CSDL1')
```

```
If @d>=5
```

```
Print 'Diem: ' + CONVERT(char(3), @d) +  
'Dat'
```

```
Else
```

```
Print 'Diem: ' + CONVERT(char(3), @d) +  
'Khong Dat'
```

2. Một số câu lệnh điều khiển



- **Cấu trúc CASE:** Được dùng để đánh giá một biểu thức và trả về một hoặc một số các kết quả dựa vào giá trị của biểu thức.
 - *Simple CASE:* Một biểu thức sẽ được dùng để so sánh với một tập các giá trị để xác định kết quả:

CASE *case_expression*

WHEN *expression* **THEN** *result*

[...n

[ELSE *else_result*

END

2. Một số câu lệnh điều khiển



- ***Searched CASE***: Đánh giá tập các biểu thức Boolean để xác định kết quả

CASE

WHEN *Boolean_expression* THEN *result*

[...*n*

[ELSE *else_result*

END

2. Một số câu lệnh điều khiển



■ Ví dụ

```
Select Masv, MaMH, Diem,  
'Nhan Xet'=Case (Diem)  
    When 0 then 'Khong dat'  
    When 1 then 'Khong dat'  
    When 2 then 'Khong dat'  
    When 3 then 'Khong dat'  
    When 4 then 'Khong dat'  
    Else 'Dat'  
End  
From Ket_Qua
```

```
Select Masv, MaMH, Diem,  
'Nhan Xet'=Case  
    When Diem<5 then 'Khong  
dat'  
    Else 'Dat'  
End  
From Ket_Qua
```



Lệnh điều khiển lặp:

■ Cú pháp

WHILE <biểu thức điều kiện>

{<câu lệnh>|**BEGIN...END**}

[BREAK]

[CONTINUE]

■ Chú ý:

- Nếu thực thi 2 hoặc nhiều câu lệnh trong mệnh đề WHILE cần bao các lệnh trong khối **BEGIN...END**

2. Một số câu lệnh điều khiển



```
declare @xh int
set @xh = 0
While @xh < (Select COUNT(masv) From Sinh_Vien)
Begin
    Set @xh = @xh + 1
    Print 'Sinh vien thu: ' + Cast(@xh
as char(3))
End
```



```
Sinh vien thu: 1
Sinh vien thu: 2
Sinh vien thu: 3
Sinh vien thu: 4
Sinh vien thu: 5
Sinh vien thu: 6
Sinh vien thu: 7
Sinh vien thu: 8
```



- Một **hàm** (Function) trong SQL Server là một chương trình con viết trên T-SQL, đảm trách một xử lý nào đó với đặc tính là sẽ nhận các tham số đầu vào và trả về một giá trị kết quả dựa trên các tham số đầu vào đã nhận.
- **Có 2 loại hàm:**
 - Hàm của hệ thống SQL Server (thuộc thư viện hàm)
 - Hàm do người dùng tạo ra



Hàm vô hướng

- Cú pháp:

CREATE FUNCTION <tên hàm>(@tên tham số kiểu dl
[= default] [,...n])

RETURNS kiểu dữ liệu trả về

[AS]

BEGIN

<Các câu lệnh SQL>

RETURN (<lệnh Select/ biểu thức vô hướng>)

END



■ Thực thi hàm

```
SELECT dbo.Tên hàm([giá trị cho biến])
```

■ *Hoặc:*

```
Print 'Thông báo' + [convert/cast]dbo.Tên hàm([giá trị cho biến])
```



■ Ví dụ 1: Hàm tính tổng số sinh viên

Create Function `fnc_tongsv()`

Returns `int`

As

Begin

Declare `@ts int`

Select `@ts=COUNT(masv)` From `Sinh_Vien`

return `@ts`

End



■ Thực thi hàm:

```
Select dbo.fnc_tongsv() as Tong_SV
```

■ Hoặc:

```
Print 'Tong so sinh vien la: ' + convert  
(char(5),dbo.fnc_tongsv())
```



■ Ví dụ 2: Trả về thứ trong tuần của dữ liệu kiểu ngày

Create function fnc_thu (@ngay date)

Returns char(10)

As

Begin

Declare @thu char(10)

Select @thu = DATENAME(dw,@ngay)

return(@thu)

End



■ Thực thi hàm:

```
Select Masv, hdem, ten, dbo.fnc_thu(ngsinh)as Thu
From Sinh_Vien
```

■ Kết quả:

Results		Messages		
	Masv	hdem	ten	Thu
1	1	Nguyen Van	An	Tuesday
2	2	Tran Viet	Anh	Sunday
3	3	Le Thanh	Huong	Monday
4	4	Le Thi	Huyen	Saturday
5	5	Le	Huyen	Saturday
6	6	Tran Le	Khanh	Wednesday
7	7	Hoang Hai	Ha	Tuesday
8	8	Nguyen Tuan	Thanh	Saturday



- Ví dụ 3: Viết hàm cho biết điểm trung bình theo mỗi môn

```
Create function fnc_DTB (@mmh char(10))  
Returns money  
Begin  
    return (select AVG(diem)  
            from    Ket_Qua  
            where MaMH=@mmh)  
End
```

- Thực thi (tham số chỉ truyền theo vị trí):

```
Print 'Diem trung binh mon CSDL1 la:' + Cast  
(dbo.fnc_DTB('CSDL1') as char(5))
```



■ Hàm trả về bảng đơn giản:

```
CREATE FUNCTION <tên hàm>(@tên tham số kiểu dl  
                        [= default] [...n] )  
  
RETURNS TABLE  
  
[AS]  
  
    RETURN (<Câu lệnh SQL>)
```

■ Thực thi:

```
SELECT <ds cột | * >  
  
FROM <dbo.Tên hàm(giá trị cho biến)>
```



- Ví dụ 4: Trả về danh sách sinh viên theo mã lớp

Create Function `fnc_DSSV (@m1 char(10))`

Returns table

As

Return

```
Select masv, hdem+' '+ten as 'Ho ten',  
Gtinh, Ma1  
From sinh_vien  
Where ma1 = @m1
```

- Thực thi hàm:

```
Select * From fnc_DSSV('CNTTAK14')
```



■ Hàm trả về bảng đa lệnh:

CREATE FUNCTION <tên hàm>(@tên tham số kiểu dl
[= default] [,...n])

RETURNS @<tên_biến_trả_về> TABLE
(<tên_cột_1> <kiểu_dữ_liệu>,
...
<tên_cột_n> <kiểu_dữ_liệu>)

[AS]

[BEGIN

<Các câu lệnh SQL>

RETURN

END]



- Ví dụ 5: *Tạo hàm trả về là Bảng danh sách sinh viên gồm: MASV, HoTenSV, TENMH, DIEM theo MaSV*

```
Create Function fnc_DSĐiem(@msv int)
Returns @DSĐiem Table (MaSV int not null, HoTen varchar(30),
TenMH varchar(30),ĐiemThi float)
As
Begin
    Insert into @DSĐiem
    Select a.MaSV, HĐem+' '+Ten as HoTen,tenMH, Điem      as
ĐiemThi
    From Sinh_Vien a join Ket_Qua b on a.MaSV=b.MaSV
    join M_Hoc c on b.MaMH=c.MaMH Where a.MaSV=@msv
Return
End
```



■ Thực thi hàm:

```
Select * From fnc_DSDiem(2)
```

■ Kết quả:

Results		Messages		
	MaSV	HoTen	TenMH	DiemThi
1	2	Tran Viet Anh	Co so du lieu 1	9
2	2	Tran Viet Anh	Tieng Anh 1	7
3	2	Tran Viet Anh	Tin Dai cuong	4
4	2	Tran Viet Anh	Tin Dai cuong	8
5	2	Tran Viet Anh	Tin dung ngan hang 1	6



■ *Sửa hàm*

ALTER FUNCTION [tên người sở hữu.]tên hàm

(@tên tham số.kiểu dữ liệu [= default] [...n])

RETURNS kiểu dữ liệu trả về | **TABLE**(đn cột | ràng
buộc cho các cột)

[**AS**] [**BEGIN**< thân hàm> **END**]

[**RETURN** [(lệnh **SELECT**)]]



Alter Function fnc_DSSV (@m1 char(10))

Returns table

As

Return

```
Select masv, hdem+' '+ten as 'Ho ten',  
Gtinh, TenL From sinh_vien join Lop on  
Sinh_Vien.MaL=Lop.MaL  
Where sinh_vien.MaL = @m1
```



■ *Xóa hàm*

DROP FUNCTION [tên người sở hữu.] tên hàm

■ *Ví dụ:*

Drop function fnc_DSSV



1. **Tạo hàm tính tuổi của các sinh viên trong bảng SV**
2. **Tạo hàm tính số tiết của mỗi môn học trong bảng Môn học (1tc=16.5 tiết)**
3. **Tạo hàm trả về số sinh viên k15 đã thi môn CSDL1**
4. **Tạo hàm trả về điểm cao nhất theo mã môn học đưa vào**
5. **Tạo hàm trả về danh sách lớp theo tên khoa**
6. **Tạo hàm trả về danh sách sinh viên và điểm của sinh viên theo tên môn học**
7. **Tạo hàm trả về số lượng sinh viên đã thi một môn học nào đó khi biết tên môn học**



- *Thủ tục (Store Procedure) là một tập các phát biểu T-SQL mà SQL Server biên dịch thành một chương trình thực thi*
- *Có 2 loại thủ tục:*
 - *Thủ tục không có tham số*
 - *Thủ tục có tham số*



■ Cú pháp

```
CREATE PROC | PROCEDURE < tên thủ tục >  
[{ @tên tham số [kiểu dl của tham số.] kiểu tham số }]  
[tùy chọn cho cursor][= default ][[ OUT | OUTPUT ][,...n ] [WITH  
<procedure_option> [...n ]]  
AS [BEGIN] < câu lệnh > [END]
```

- **[= default]**: gán giá trị mặc định cho tham số. Nếu không gán giá trị mặc định thì tham số nhận giá trị NULL.
- **OUT | OUTPUT**: chỉ định tham số đó là tham số xuất (không dùng được với kiểu dữ liệu Text và image).
- **[,...n]**: chỉ định rằng có thể khai báo nhiều tham số.



■ Thực thi

EXEC | EXECUTE <tên thủ tục >

@tênbiến = giá trị | @biến chứa tham số [**OUTPUT**]
| [**DEFAULT**] } [,...n]

[**Print** <thông báo> [+ < tên biến kiểu dl của biến>]]

4. Thủ tục



- ***Ví dụ 1: Tạo thủ tục có tên: Pro_DSSV với các thông tin: MaSV, HoTen, MaL***

Create PROC Pro_DSSV
AS

Select MaSV, HDem + Ten As HoTen, MaL
From Sinh_Vien

- ***Thực thi***

EXECUTE Pro_DSSV

100 %

Results Messages

	MaSV	HoTen	MaL
1	1	Nguyen VanAn	CNTTAK14
2	2	Tran VietAnh	CNTTAK14
3	3	Le ThanhHuong	CNTTBK14
4	4	Le ThiHuyen	TABK14
5	5	Le Huyen	TABK14
6	6	Tran LeKhanh	TABK14
7	7	Hoang HaiHa	CNTTBK13
8	8	Nguyen Tuan...	CNTTBK13
9	9	Vu HuyenTrang	CNTTAK14

4. Thủ tục



■ Ví dụ 2: Lập danh sách sinh viên theo mã lớp

```
Create PROC Pro_DS @ML char(10)
```

```
As
```

```
Begin
```

```
    Select MaSV, HDem, Ten, MaL
```

```
    From Sinh_Vien
```

```
    Where MaL=@ML
```

```
End
```

■ Thực thi (truyền tham số theo tên)

```
EXEC Pro_DS @ML = 'CNTTBK13'
```

Hoặc (truyền tham số theo vị trí)

```
EXEC Pro_DS 'CNTTBK13'
```



- ***Ví dụ 3: Tạo thủ tục trả về tổng số sinh viên có trong bảng sinh viên.***

```
Create Proc Pro_SlgSV @sl int output  
AS
```

```
    Select @sl = COUNT(Masv) From Sinh_Vien
```



■ *Thực thi thủ tục (truyền theo tên)*

```
Declare @tong int
```

```
EXEC Pro_slgs_v @sl = @tong output
```

```
Print 'So luong SV hien co la: ' + Cast(@tong as  
char(10))
```

■ *Thực thi thủ tục (truyền theo vị trí)*

```
Declare @tong int
```

```
EXEC Pro_slgs_v @tong output
```

```
Print 'So luong SV hien co la: ' + Cast(@tong as  
char(10))
```



- *Ví dụ 4: Viết thủ tục bổ sung thêm bản ghi cho bảng môn học*

Create Proc Pro_nhậpMH

@mmh char(10), @tmh varchar(30), @stc int, @mk
char(10)

As

Insert into M_Hoc

Values (@mmh, @tmh, @stc, @mk)



■ *Gọi thực thi thủ tục:*

```
EXEC Pro_nhậpMH 'EBK', 'E-Banking', 3, 'CNTT'
```

■ *Hoặc:*

```
EXEC Pro_nhậpMH @mmh='EBK', @tmh='E-Banking',  
@stc=3, @mk='CNTT'
```



Thay đổi thủ tục

■ Cú pháp:

```
ALTER PROC | PROCEDURE < tên thủ tục> [{@tên  
tham số [kiểu dl của tham số.] kiểu tham số }] [tùy chọn  
cho cursor][= default ][[ OUT | OUTPUT ][,...n ] [WITH  
<procedure_option> [...n ]  
AS [BEGIN] < câu lệnh> [END]
```



■ Ví dụ:

```
Alter PROC Pro_DS @ML char(10)
```

```
As
```

```
Begin
```

```
    Select MaSV, HDem+' '+Ten as Ho_Ten,  
    MaL From Sinh_Vien Where MaL=@ML
```

```
End
```



- **Xóa thủ tục**

- **Cú pháp:**

DROP PROC | PROCEDURE [tên người sở hữu.] tên_thủ_tục

- **Ví dụ:**

Drop PROC Pro_DS



1. Tạo/thực thi thủ tục cho biết danh sách kết quả sinh viên theo kỳ thi và theo mã sinh viên
2. Tạo/thực thi thủ tục nhập dữ liệu cho bảng sinh viên
3. Viết/thực thi thủ tục kiểm tra xem có tồn tại một môn học theo mã môn học đưa vào hay không rồi đưa ra thông báo
4. Viết/thực thi thủ tục cho biết sinh viên đầy thuộc lớp nào, khoa nào theo mã sinh viên nhập vào
5. Viết/thực thi thủ tục cho biết danh sách các sinh viên đã qua (điểm ≥ 5) môn học nào đó khi đưa vào tên môn học
6. Viết/thực thi thủ tục trả về tổng số môn đã học theo mã sinh viên nhập vào.



- *Trigger là một kiểu Stored Procedure đặc biệt (không có tham số, không có giá trị trả về) được thực thi một cách tự động khi xảy ra một sự kiện trên Database server (chèn, cập nhật hoặc xóa trong một bảng).*
- *Trigger được sử dụng một cách phổ biến để ép các thao tác trên CSDL tuân theo các quy tắc nhằm đảm bảo tính toàn vẹn và tính nhất quán của dữ liệu.*



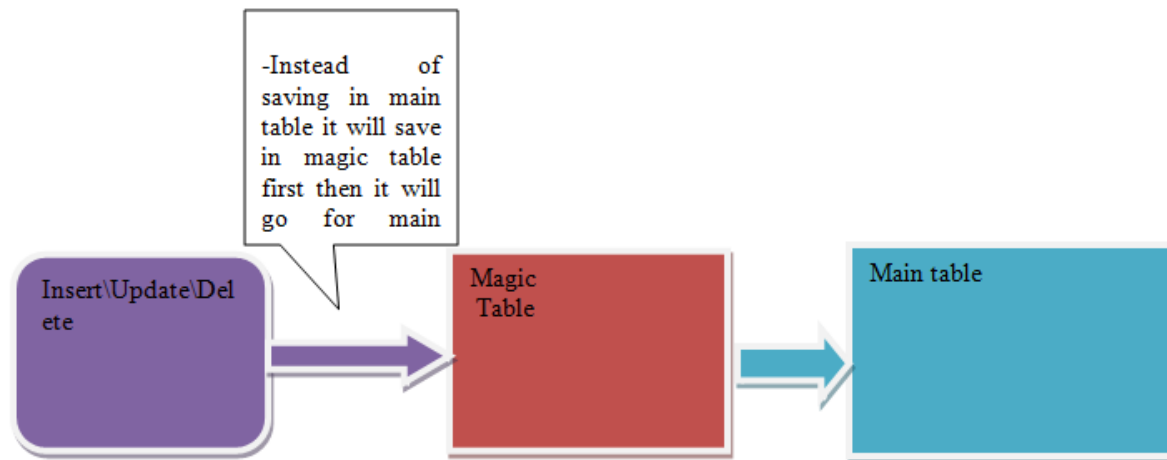


- *Một Trigger được định nghĩa trên một bảng nhưng các xử lý trong Trigger có thể sử dụng nhiều bảng khác*
- *Trigger được chia ra làm 2 nhóm:*
 - **DDL (Data Definition Language) triggers** được thực thi đáp ứng các sự kiện định nghĩa lược đồ dữ liệu như: **CREATE, ALTER, DROP.**
 - **DML (Data Modification Language) triggers** thực thi khi một người sử dụng sửa đổi dữ liệu thông qua **INSERT, UPDATE, DELETE** trên Bảng hoặc View

5. Tạo bể dữ liệu - Trigger

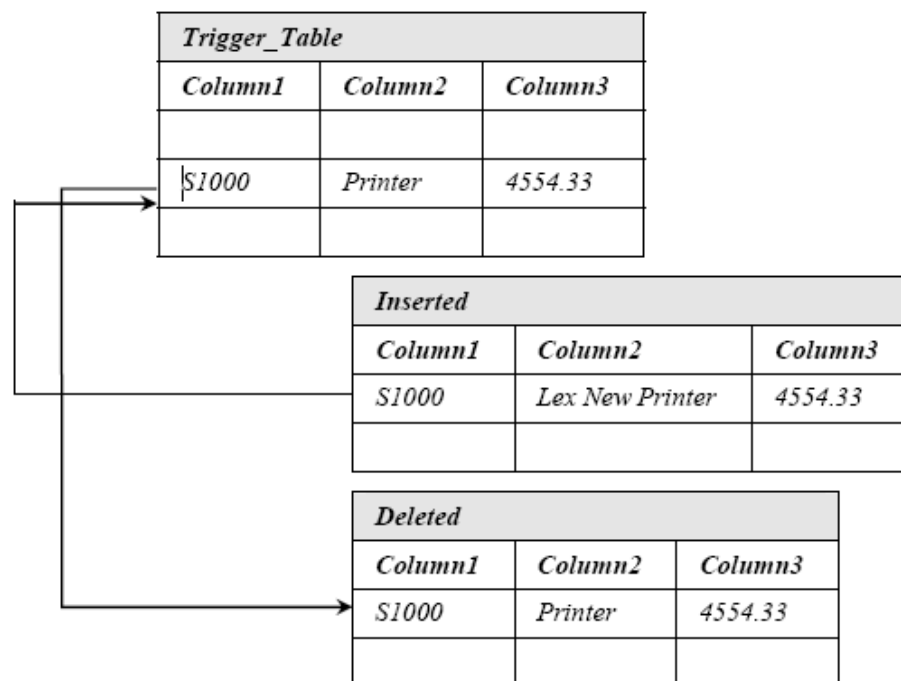


- Trigger sẽ tự động thực hiện khi có bản ghi được Insert/Update/delete ở bảng gắn với nó. Khi đó, SQL server tạo ra hai bảng tạm Inserted và Deleted (*magic tables*)





- **Chú ý:** đối với UPDATE trigger thì SQL Server sẽ tạo cả hai bảng: INSERTED và DELETED
- UPDATE = DELETE dòng cũ + INSERT dòng mới**





■ Tạo Trigger

CREATE TRIGGER <tên trigger>

ON <tên bảng>|<tên view> [**WITH ENCRYPTION**]

FOR|AFTER|INSTEAD OF <**INSERT|UPDATE| DELETE**>

AS [**IF UPDATE** (column list)][**AND|OR** <**UPDATE** (column)>]

<câu lệnh T- SQL>



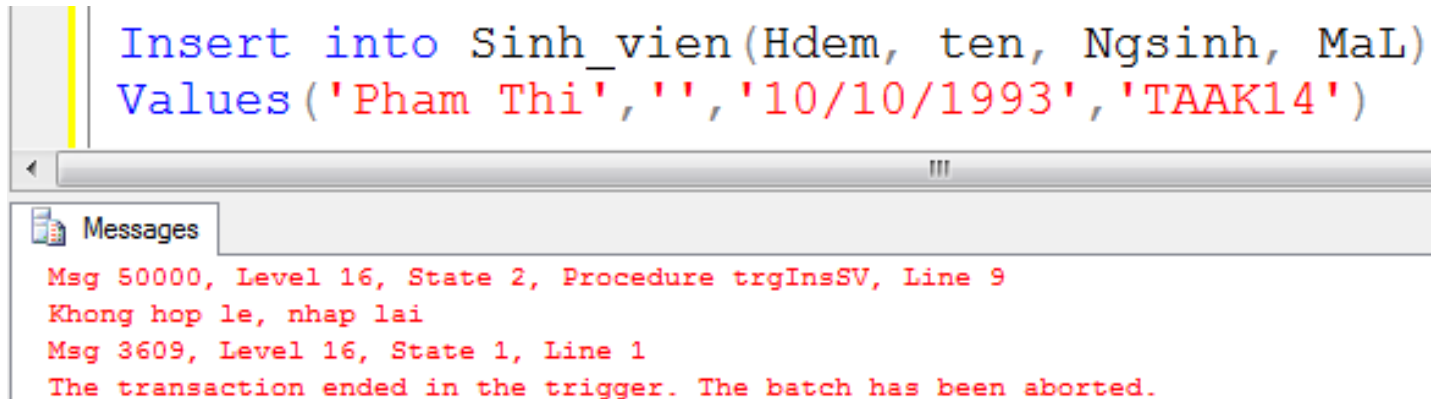
- **WITH ENCRYPTION**: ngăn chặn người dùng khác xem văn bản của trigger.
- **FOR/ AFTER** : trigger được thực thi sau khi tất cả các câu lệnh SQL gây ra.
- **INSTEAD OF**: trigger được thực thi thay cho các câu lệnh SQL gây ra trigger (thường dùng cho view).
- **INSERT | UPDATE | DELETE**: xác định câu lệnh mà khi thực thi các thao tác này trên bảng hoặc view sẽ gây ra trigger.
- **IF UPDATE** (column list): dùng để kiểm tra hành động Insert, Update (không dùng cho Delete) các giá trị trên các cột.

5. Tạo bảng dữ liệu - Trigger



```
Create Trigger trgInsSV
On sinh_vien
For Insert
As
    Declare @ten varchar(10)
    Select @ten = ten From Inserted
    If @ten=''
    Begin
        Raiserror ('Khong hop le, nhap lai',16,2)
        Rollback tran
    End
```

Thực hiện thêm dữ liệu



```
Insert into Sinh_vien(Hdem, ten, Ngsinh, MaL)
Values('Pham Thi',' ','10/10/1993','TAAK14')
```

Messages

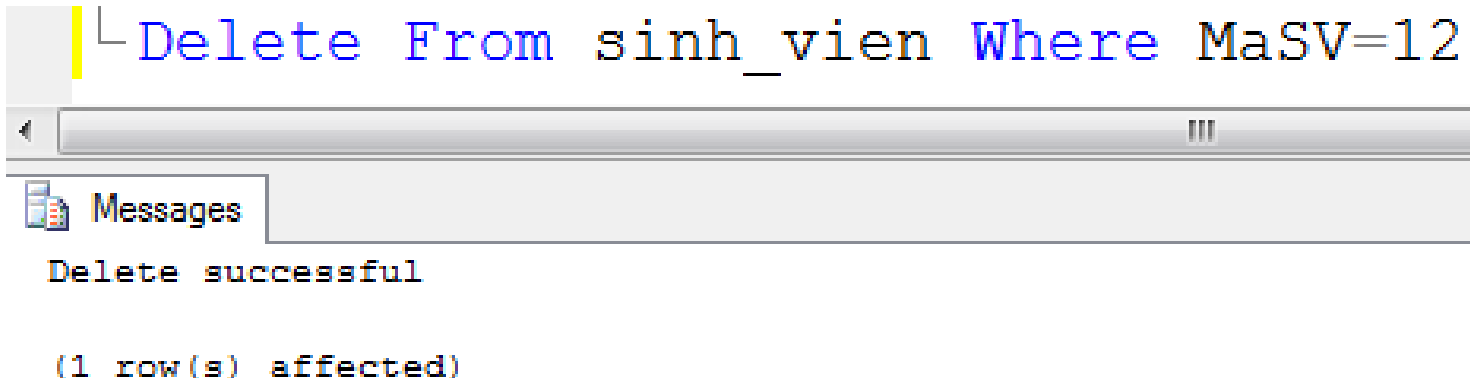
Msg 50000, Level 16, State 2, Procedure trgInsSV, Line 9
Khong hop le, nhap lai

Msg 3609, Level 16, State 1, Line 1
The transaction ended in the trigger. The batch has been aborted.



```
Create trigger trgDelSv  
on sinh_vien  
After Delete  
As  
    Print 'Delete successful'
```

Khi xóa dữ liệu trong bảng ***SINH_VIEN***



```
Delete From sinh_vien Where MaSV=12
```

Messages

```
Delete successful
```

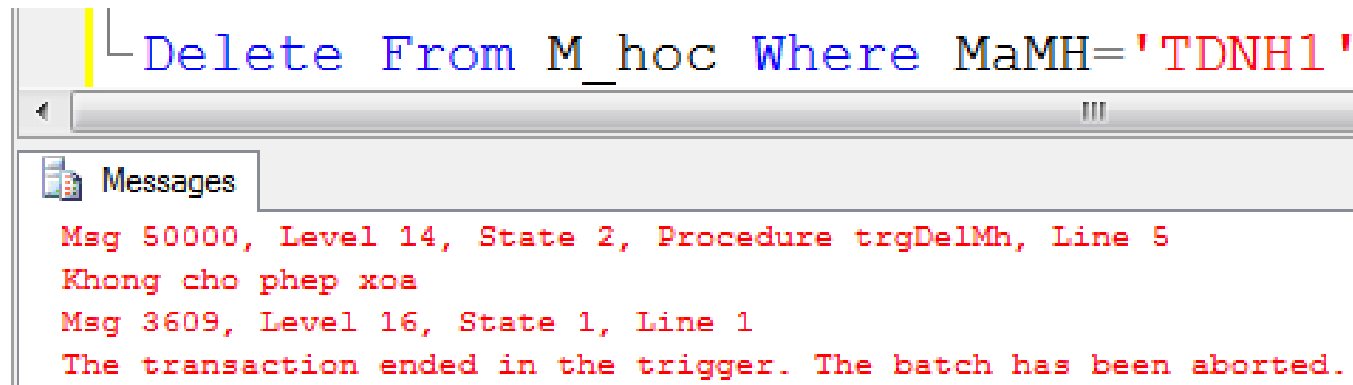
(1 row(s) affected)

5. Tạo bảng dữ liệu - Trigger



```
Create trigger trgDelMh  
on M_Hoc  
For Delete  
As  
    Raiserror ('Khong cho phep xoa', 14,2)  
Rollback tran
```

Khi xóa dữ liệu trong bảng M_Hoc



5. Tạo bảng dữ liệu - Trigger



```
Create trigger trgUpTl
on Lop
For update
As
    Declare @tl varchar(30)
    Select @tl = TenL from inserted
    If (Select count(*) From Lop Where @tl=TenL)>1
    Begin
        raiserror ('Trung',12,1)
        Rollback tran
    End
```

Thực hiện Update trên bảng LOP với tên lớp đã có

```
Update Lop set TenL='Tieng Anh B K14' Where MaL='TAAK14'
```

 Messages

```
Msg 50000, Level 12, State 1, Procedure trgUpTl, Line 9
Trung
Msg 3609, Level 16, State 1, Line 1
The transaction ended in the trigger. The batch has been aborted.
```



- *Tạo trigger Delete xóa bản ghi trong bảng sinh viên thì cũng xóa các bản ghi tương ứng trong bảng kết quả*

```
Create trigger trgDelSvKq
On Sinh_vien
instead of Delete
As
Begin
    Delete From Ket_qua
    Where MaSV=(Select Masv From deleted)
    Delete From Sinh_vien
    Where MaSV=(Select Masv From deleted)
End
```

5. Tạo dữ liệu - Trigger



Select * from Sinh_Vien

	MaSV	HDem	Ten	NgSinh	QQuan	GTinh	MaL	EMail
1	1	Nguyen Van	An	1993-10-12	Hai Phong	Nam	CNTTAK14	AnNv@gmail.com
2	2	Tran Viet	Anh	1993-12-12	Hai Duong	Nam	CNTTAK14	NULL
3	3	Le Thanh	Huong	1992-02-10	Hai Duong	Nam	CNTTBK14	NULL
4	4	Le Thi	Huyen	1992-02-15	Ha Noi	Nu	TABK14	HuyenLT@gmail.com
5	5	Le	Huyen	1992-02-15	Ha Noi	nu	TABK14	huyenle@gmail.com
6	6	Tran Le	Khanh	1992-05-20	Ha Noi	Nu	TABK14	Khanhlt@gmail.com
7	7	Hoang Hai	Ha	1992-02-11	NULL	Nam	CNTTBK13	Hahh@yahoo.com
8	8	Nguyen Tuan	Thanh	1991-10-12	Nghe An	Nam	CNTTBK13	NULL

Bản ghi muốn xóa

Select * from ket_qua

	MaMH	MaSV	Diem	KyThi
1	CSDL1	1	9	3
2	CSDL1	2	9	3
3	CSDL1	3	4	3
4	CSDL1	6	7	3
5	TA1	1	9	1
6	TA1	2	7	1
7	TA2	4	9	3
8	TDC	1	8	1
9	TDC	2	4	1
10	TDC	2	8	3
11	TDC	3	8	1
12	TDC	6	0	1
13	TDC	6	0	2
14	TDN...	2	6	4

5. Tạo dữ liệu - Trigger



Delete From sinh_vien
Where Masv=3

Select * from Sinh_Vien

	MaSV	HDem	Ten	NgSinh	QQuan	GTinh	MaL	EMail
1	1	Nguyen Van	An	1993-10-12	Hai Phong	Nam	CNTTAK14	AnNv@gmail.com
2	2	Tran Viet	Anh	1993-12-12	Hai Duong	Nam	CNTTAK14	NULL
3	4	Le Thi	Huyen	1992-02-15	Ha Noi	Nu	TABK14	HuyenLT@gmail.com
4	5	Le	Huyen	1992-02-15	Ha Noi	nu	TABK14	huyenle@gmail.com
5	6	Tran Le	Khanh	1992-05-20	Ha Noi	Nu	TABK14	Khanhtl@gmail.com
6	7	Hoang Hai	Ha	1992-02-11	NULL	Nam	CNTTBK13	Hahh@yahoo.com
7	8	Nguyen Tuan	Thanh	1991-10-12	Nghe An	Nam	CNTTBK13	NULL

Select * from ket_qua

	MaMH	MaSV	Diem	KyThi
1	CSDL1	1	9	3
2	CSDL1	2	9	3
3	CSDL1	6	7	3
4	TA1	1	9	1
5	TA1	2	7	1
6	TA2	4	9	3
7	TDC	1	8	1
8	TDC	2	4	1
9	TDC	2	8	3
10	TDC	6	0	1
11	TDC	6	0	2
12	TDNH1	2	6	4



1. Tạo trigger đảm bảo khi nhập bản ghi cho bảng Khoa thì thuộc tính tên khoa không được rỗng
2. Tạo trigger không cho xóa bản ghi trong bảng Lớp
3. Tạo trigger không cho sửa thuộc tính điểm trong bảng Kết quả
4. Tạo trigger thêm bản ghi cho bảng Kết quả với Kỳ thi phù hợp (với 1 môn thi của 1 sinh viên thì kỳ thi sau phải lớn hơn kỳ thi trước)
5. Tạo trigger đảm bảo mỗi lần thêm một sinh viên của lớp nào thì số lớp đó tăng thêm 1



■ Sửa trigger

ALTER TRIGGER <tên trigger>

ON table | view

FOR | AFTER [**INSERT** | **UPDATE** | **DELETE**]

AS <câu lệnh T- SQL>



■ Ví dụ:

```
Alter trigger trgDelMh
on M_Hoc
For Delete
As
Declare @tm varchar(30)
Select @tm=TenMH From deleted
If @tm is not null
Begin
    Raiserror ('Không cho phép xóa', 14,2)
    Rollback tran
End
```



- Chèn thêm một bản ghi mới cho bảng M_Hoc có tên môn học là null:

Insert into M_Hoc (MaMH, MaKH, SoTC) values ('MangTT', 'CNTT', 3)

```
Delete from M_Hoc where MaMH='MangTT'
```

100 % <

Messages

(1 row(s) affected)

```
Delete from M_Hoc where MaMH='CSDL2'
```

100 % <

Messages

Msg 50000, Level 14, State 2, Procedure trgDelMh, Line 9
Không cho phép xóa
Msg 3609, Level 16, State 1, Line 1
The transaction ended in the trigger. The batch has been aborted.



■ Xóa trigger

```
DROP TRIGGER <tên trigger > [ ,...n ]
```

■ Ví dụ:

```
Drop trigger trgDelSv
```

